软件测试课程笔记整理（全 Slides 汇总）

> 本笔记基于工作区全部 26 份幻灯片（PDF）从头到尾整理而成，并结合最后一份复习文件《软件测试-复习.pdf》（模糊测试设计）对重点内容进行标注。

标注图例

标记	含义
⭐ 重点	核心概念/定义，考试高频考点
📌 复习关联	与期末复习文件（模糊测试设计）直接相关的内容
💡 易考/思考点	幻灯片中留的思考题、讨论题，常作为考题来源
⚙️	代码/公式细节

目录

- 第 0 部分：课程总览（软件测试01）
- 第 1 部分：测试基础概念（第1章）
  - 1.1 常用测试术语
  - 1.2 常用测试分类
  - 1.3 测试分析框架
- 第 2 部分：经典测试程序（1.2 测试设计与测试策略）
  - 2.1 三角形程序 Triangle
  - 2.2 日期程序 NextDay
  - 2.3 均值方差程序 MeanVar
- 第 3 部分：软件 Bug 与 PIE 模型（第2章）
  - 3.1 PIE 模型
  - 3.2 Bug 的反向定义
  - 3.3 Bug 的不确定性
  - 3.4 Bug 的非单调性
  - 3.5 Bug 间的干涉性

- 第 4 部分：随机测试方法（3.1 章）
  - 4.1 简单随机测试
  - 4.2 自适应随机测试 ART
  - 4.3 引导性随机测试（符号执行）
- 第 5 部分：等价类测试方法（3.2 章）
  - 5.1 软件等价类划分
  - 5.2 划分随机测试方法
- 第 6 部分：组合测试方法（3.3 章）
- 第 7 部分：边界故障假设（4.1 章）
  - 7.1 黑盒边界故障假设
  - 7.2 白盒边界故障假设
- 第 8 部分：变异缺陷假设（4.2 章）
  - 8.1 变异分析基本概念
  - 8.2 变异测试优化技术
- 第 9 部分：逻辑故障假设（4.3 章）
  - 9.1 逻辑测试基础
  - 9.2 逻辑故障结构
- 第 10 部分：Web 环境搭建（实验课，基于 Java）
- 第 11 部分：期末复习——模糊测试设计（结合复习文件）

第 0 部分：课程总览（软件测试01）

课程目标：工业场景下的自动化、智能化测试技术。 **课程内容**：源代码测试、移动应用测试、智能软件测试。 **课程考核**：Survey 方向 + 工具实现方向 + 期末考试；课堂报告、课堂小测试、课后小测试。

测试的基本问题（图灵测试的启发）⭐

图灵测试诱导出软件测试的三大基本问题：

1. **测试生成问题**（应该问什么问题？）
2. **测试预言问题**（什么是正确答案？）
3. **测试终止问题**（问多少问题够了？）

自动化软件测试六大方向

自动化测试生成、自动化测试执行、自动化测试优化、自动化测试修复、自动化测试扩增、自动化结果分析。

软件缺陷的触发与传播机理 ⭐

- **Fault（故障）**：静态存在于程序中的缺陷代码（defect）。
- **Error（错误）**：运行缺陷代码后导致的错误内部状态。
- **Failure（失效）**：错误状态传播到外部、与预期行为不符的表现。

> 例：MeanVar 程序 `for (int i = 1; i < length; i++)`（应为 i=0）
>
> - 输入 [3,4,5]：sum=4+5=9（错）→ mean=3（错）→ 观察到输出错误（Failure）。
> - 输入 [0,4,5]：sum=4+5=9，mean=9/3=3 = 正确值 3 → **执行了故障却不产生 Error！**
> - 修正 `length-1` 后输入 [3,5,4]：sum=8，mean=4 = 正确 → **Error 未传播为 Failure！**

PIE 模型 ⭐

故障要被发现必须满足三步：

1. **Execution（执行）**：包含故障的语句必须被执行到；
2. **Infection（感染）**：程序状态必须出错；
3. **Propagation（传播）**：错误状态必须传播到输出。

> 递进关系：满足(3)⇒满足(2)⇒满足(1)；非必然关系：满足(1)未必满足(2)，满足(2)未必满足(3)。

Bug 空间聚集

William Dickinson et al. @ FSE 2001：**故障通常聚在一起**（在输入/剖面空间中呈簇状分布）。

测试的局限性（三大挑战）⭐

1. **输入空间庞大**：无法穷举所有输入；
2. **实现逻辑复杂**：无法想到所有场景；
3. **测试预言未知**：无法判定测试的预期输出。

随机测试与模糊测试 📌

- **随机测试**：基于大数定律，测试执行次数够多 + 测试数据随机生成 → 概率低的偶然现象（Bug）发生。

- 发现的 Bug 类型：错误输出、无效输入、断言失败、异常崩溃。
- **模糊测试（Fuzzing）**：通过异常的（非预期）输入自动化发现待测程序缺陷的技术。⭐（复习文件核心！）
    - 包含环节：**数据生成、测试运行、结果反馈**。

经典测试技术（三大经典技术）⭐

技术	解决什么问题	核心思想
变异测试	测试输入如何生成	找出有效的测试用例，发现程序中真正的错误
蜕变测试	测试预言如何获取	蜕变关系 MR：多次执行目标程序时输入与输出间期望遵循的关系（如 sin 的 6 条 MR）
差分测试	测试结果如何判断	同一测试用例运行到一系列相似功能应用中，观察执行差异检测 Bug

智能软件测试（了解）

- 智能软件质量挑战：任务目标难以精确表达、非确定性开放环境、复杂软硬件场景（Uber 自动驾驶事故、Alexa 误报警、Yelp 修复助手）。
- **结构化覆盖测试准则**（DeepGauge）：神经元覆盖 NAC、多节神经元覆盖 KMNC、神经元边界覆盖 NBC、强神经元覆盖 SNAC、top-k 神经元覆盖等。
- 面向智能软件的测试：等价类测试、边界值测试（对抗样本）、群体智能测试（众包）、蜕变测试（电磁信号 ElecDaug、医疗 TauMed）、深度学习框架差分模糊测试（Duo、Predoo、GraphFuzz）等。

第 1 部分：测试基础概念（第1章）

1.1 常用测试术语

什么是测试

测试是通过设计实验性操作，观察目标对象的行为或输出，以发现缺陷、验证功能或评估性能的方法。核心目的：**发现缺陷、验证功能或非功能属性**。

> Dijkstra：“Program testing can be used to show the presence of bugs, but never to show their absence!”（测试只能证明 Bug 存在，不能证明不存在）

测试用例 ⭐

测试用例 tc 是一个三元组 `<t, o, θ>`：

- **t**：测试数据/测试输入（文字、交互、图像、语音、VR 输入等）；
- **o**：测试预言/预期输出（判断输出结果是否正确的参考依据）；
- **θ**：测试环境（运行环境和状态要求）。

测试集（测试套件）T = {tc1, …, tcn}。 使用方式变化：`<t, o, θ>` → `<t, o>` → `<t>`（越来越弱）。

测试数据

- 为测试执行准备的输入数据，可作为功能输入验证输出，也能触发异常场景。
- 应尽可能接近真实执行数据的分布和特征；可由测试人员设计或工具产生。
- 测试数据质量决定测试质量：不全→遗漏场景；过多无效→成本增加。

测试预言 ⭐（复习关联：评估准则）

- **定义**：一种用于判断程序在给定测试输入下执行结果是否符合预期行为的方法，定义了预期输出。
- **作用**：将预期输出与真实输出比较，验证被测软件正确性。
- **影响**：测试预言质量直接影响测试有效性——高质量预言准确判断正确性与缺陷。

测试环境

- 定义：完成软件测试所必需的硬件、软件、外围设备和历史数据的总称。
- 组成：服务器、操作系统、数据库、中间件、网络设备、测试工具等。
- 不稳定原因：部署不合理、数据被修改、服务器宕机或升级等。

测试报告与缺陷报告

- 测试报告至少包含四部分 `<θ, t, o, d>`：θ 测试环境、t 测试输入、o 测试输出（截图/视频）、d 结果描述。
- 相关术语：测试需求、测试用例、测试脚本、测试报告、缺陷报告、测试汇总报告。

待测软件（SUT）⭐

- 待测软件 SUT = 待测程序 P + 相关文档。
- P 是有序元素集合 {u1, …, um}：源代码→代码行；字节码→指令行；二进制→比特码；应用程序→代码/接口/服务/界面单元。
- 软件制品类型：Web 应用、移动应用、嵌入式应用。

1.2 常用测试分类

三种分类维度：

1. **源代码信息依赖程度**：白盒 / 灰盒 / 黑盒；
2. **软件开发流程**：单元 / 集成 / 系统 / 验收测试；
3. **软件质量属性**：功能 / 性能 / 安全 / 兼容性 / 易用性测试。

白盒测试（White-box）⭐

- 依赖源代码信息，测试应用程序内部结构和运行情况；
- 从程序内部角度设计测试用例，通过程序路径覆盖发现缺陷；
- 需理解内部结构，成本高昂。

黑盒测试（Black-box）⭐

- 从用户角度出发，针对界面、功能及外部结构测试，不考虑内部逻辑；
- 依据软件规范/规格说明/设计文档设计；
- 优点：应用范围广、测试效率高、覆盖全面。

灰盒测试（Gray-box）

- 介于白盒与黑盒之间：注重内部逻辑，但源代码信息不像白盒详尽；
- 通过表征性现象/事件反映内部运行状态，用内部间接信息设计测试。
- ⚠️ 灰盒 ≠ 白盒 + 黑盒。

按软件开发流程分类（V-Model）⭐

- **单元测试**：最小可测试单元（函数/过程）；基本原则——测试用例相互独立；一般由开发人员实施。
- **集成测试**：将已测单元组装成模块/子系统/系统，确认单元间交互无问题。
- **系统测试**：将软硬件看作整体，检验系统是否符合预期；包括功能、性能、安全、兼容性、稳定性测试。
- **验收测试**（交付测试）：验证是否满足验收标准，由用户或授权机构决定是否接受；分内部验收与外部验收，可采用众包方式。

软件质量属性

产品质量属性四大特征（Brooks, No Silver Bullet）：**复杂性、顺从性、易变性、不可见性**。

- 功能测试：验证功能正确、发现缺陷、增强信心（黑盒、基于需求、可重复、独立）。
- 性能测试：响应时间、吞吐量、并发、资源利用率；容量规划。
- 安全测试：漏洞扫描、渗透测试、安全配置、身份验证、授权机制；静态/动态分析。

- 兼容性测试：硬件/操作系统/浏览器/数据兼容；适配性测试（Adaptability）。
- 易用性测试：界面测试、交互测试、用户体验测试。

1.3 测试分析框架

软件工程三大问题 ⭐

- S 问题（文档生成）、P 问题（代码生成）、T 问题（测试生成）。

软件质量定义 ⭐

给定待测程序 P 和规格需求 S，软件质量 Q 定义为： **$Q(P, S) = (P \cap S) / (P \cup S)$**

- $P \cap S$：行为一致性（质量基石）；
- $S - P$：未被程序满足的规格需求；
- $P - S$：程序中超出规格的部分（过度实现，可能有利也可能有害）。
- 隐含需求：需求规范可能不完备或不正确。

> 例（Triangle）：规格 S 要求 a,b,c 为 1~100 整数；程序 P 未实现该检查（S−P），却过度实现了"任意两边之和大于第三边"（P−S）。

Gourlay 测试分析框架 ⭐

- 基本框架：`<P, S, T>` —— P 一组程序、S 一组规格、T 一组测试。
- **扩展框架：** `<P, S, T, O, O>`：$O \subseteq P \times S$ 是**通用测试预言**（理论上 P 相对 S 正确），$O \subseteq P \times S \times T$ 是**实际测试预言**（在测试 t 条件下被判定正确）。
- 测试维恩图七区域：区域 1（同属 S、P、T）、区域 2（$(S \cap P) - T$ 未测试的指定行为）、区域 3（$T \cap (P - S)$ 测试了未指定行为）、区域 4（$T \cap (S - P)$ 测试了未实现行为）等。

测试预言的性质 ⭐

- **正确性**：O(P,S,t) 成立 ⇒ 程序确实正确执行；
- **完备性**：程序确实正确执行 ⇒ O(P,S,t) 成立；
- **完美性**：正确性 + 完备性 ⇔ corr(P,S,t) ⇔ O(P,S,t)。
- 难点：测试集 T 难以穷尽 → 完美性难以实现；预言过精确 → 程序实际正确但预言不通过（如计算精度）。

测试方法 / 测试准则定义 ⭐

- 测试方法 M 是函数：$M: P \times S \to T$（基于程序 P 和规格 S 生成测试）。

- 测试准则 TC ⊆ P×S×2^T，与程序、规格、测试集的关系；充分性可扩展为测试集+测试预言的配对充分性。
- 准则蕴含：2^Tc1 ⊆ 2^Tc2 记为 C1 ≥ C2（满足 C1 的测试集一定满足 C2）；偏序关系，不是全序；是经验关系，不是理论关系。
- 测试预言强弱：∀t∈T, O1(P,t) ⇒ O2(P,t)，则 O1 比 O2 强（O1 ≥ O2）；等价 O1 ≡ O2。

第 2 部分：经典测试程序（1.2 测试设计与测试策略）

> 三个贯穿全课程的经典程序：Triangle、NextDay、MeanVar，用于演示各类测试方法。

2.1 三角形程序 Triangle

需求规格（输入 1~100）⭐

1. a, b, c 为 1 到 100 的整数，否则输出"无效输入值"；
2. 任意两边之和大于第三边，否则输出"无效三角形"；
3. 三边相等输出"等边三角形"；
4. 两边相等输出"等腰三角形"；
5. 其他情况输出"普通三角形"。

Python 实现

```python
def triangle(a, b, c):
    if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):
        print("无效输入值")
    elif not (a + b > c and b + c > a and c + a > b):
        print("无效三角形")
    elif a == b == c:
        print("等边三角形")
    elif a == b or b == c or c == a:
        print("等腰三角形")
    else:
        print("普通三角形")
```

不同语言对测试的影响（思考）

- Python：动态类型，注意不同类型数据组合与边界；依赖库版本（虚拟环境隔离）。
- C：指针、内存管理、字节序、不同编译器行为差异。
- Java：面向对象、严格类型检查、JDK 版本一致性、Maven/Gradle 依赖管理。

- 仓颉：静态强类型、编译期类型检查、丰富测试工具链。

💡 思考：相同需求规格的不同代码实现（如嵌套 if 与 elif 的写法）带来测试设计差异性；训练分类器/神经网络实现三角形分类需要多少样本？

2.2 日期程序 NextDay

需求规格 ⭐

输入 year、month、day 三个变量，输出下一天日期：

1. $1 \le year \le 9999$；
2. $1 \le month \le 12$；
3. $1 \le day \le 31$；
4. 输出日期为下一天。

闰年规则（隐含需求）⭐

- 一年约 365.2422 天；每四年一次闰年，但**世纪年需是 400 的倍数**。
- 例：1980、1996、2000 是闰年；1900 不是闰年。

```
def LeapYear(year):
    if year % 400 == 0: return 1
    if year % 100 == 0: return 0
    if year % 4 == 0:   return 1
    else:               return 0
```

next_day 实现要点

- 月份分类：1/3/5/7/8/10（31 天）、4/6/9/11（30 天）、12（跨年）、2（闰年 29 天/平年 28 天）。
- 边界：day=28 且闰年 → 29；day=29 → 下月 1 日；day=31 → 跨月。
- 💡 讨论：能否用有限测试确定一个 NextDay 程序？如何测试农历与公历转换程序？
- 测试策略：分支+计算的结合、等价类与边界值结合、控制流与数据流结合。

2.3 均值方差程序 MeanVar

需求规格

- 输入长度为 L 的数组 X，返回平均值 mean 和方差 var；
- 隐含需求：X[i] 为实数；$L \ge 1$。
- ⚙️ $mean = \Sigma xi/n$；$var = \Sigma(xi-mean)^2/n$；修正方差分母为 $n-1$。

两种方差计算公式（稳定性问题）⭐

1. 基本公式：$var = \Sigma(xi-mean)^2/n$ —— 直观但计算量大；
2. 快捷公式：$var = \Sigma xi^2/n - mean^2$ —— 效率高但**数值不稳定**（大数减小数，精度损失）。
3. **增量式算法（Welford）**：数值稳定，适用于在线/增量数据。

```
double variance(double data[], int n) {
    double mean = 0, M2 = 0;
    for (int i = 0; i < n; i++) {
        double x = data[i];
        double delta = x - mean;
        mean += delta / (i + 1);
        M2 += delta * (x - mean);
    }
    return M2 / (n - 1);
}
```

- 💡 课后实验：X[i] 数值很大但彼此很接近（偏差小）时对比不同方差算法的精度。
- 测试策略：循环结构对测试的影响、数值计算对测试的影响、不同算法对测试的影响。

第 3 部分：软件 Bug 与 PIE 模型（第2章）

3.1 PIE 模型

Bug 的定义（IEEE 1044-2009 相关）⭐

术语	定义	观察难度
Fault（故障）	静态存在于程序中的缺陷代码	需经验丰富的程序员代码审查
Error（错误）	程序运行缺陷代码后导致的错误状态	需监控运行信息、查看日志
Failure（失效）	错误状态传播到外部被感知的现象	测试人员对比输出即可判定

- Problem：用户对软件运行结果不满意的情况；Defect：产品中不符合要求的缺陷。

PIE 模型（触发与传播机理）⭐

1. **Execution**：含故障的语句被执行；
2. **Infection**：程序状态变错；
3. **Propagation**：错误状态传播到输出。

> 例：MeanVar 程序第 4 行 `for (int i = 1; ...)`（应为 i=0）为 Fault； 输入 [3,4,5] 时 sum=9（应为 12）产生 Error；输出 mean=3（应为 4）为 Failure。 输入 [0,4,5] 时 sum=9，mean=3=正确值 → 不产生 **Error**（执行了故障但测试通过！）。

- ⚠️ PIE 模型通常用于传统软件，不完全适用于深度学习等智能软件系统缺陷。

3.2 Bug 的反向定义

正向定义 vs 反向定义 ⭐

- **正向定义**：给定完全正确的程序 P 和测试 t，若修改版本 P' 失效，则 P'\P 为故障。
- **反向定义**（实践中常用）：给定程序 P 和测试 t，若 P(t) 失效，修改后得到 P'，若 P'(t) 通过，则确认修改部分 P\P' 为故障。

反向定义的应用与风险

- 应用：通过**失效的消失**确认故障位置，依赖测试和修复过程。
- 例：MeanVar 输入 [4,3,5] 预期 4，实际 3.5。修复方案 1：i=1→i=0；修复方案 2：L→L−1。两种都能让该测试通过，但只有方案 1 是正确修复！
- ⚠️ 风险：修复依赖测试集与修复方式，初级程序员可能错误修复引入新 Bug。
- 反向定义流程：执行测试 → 失效分析 → 缺陷定位 → 缺陷修复 → 测试确认。

3.3 Bug 的不确定性

定义 ⭐

给定待测程序 P 和测试 t，P(t) 失效。若不同修复方法得到两个程序 P1 ≠ P2，均使 t 通过，则 P\P1 和 P\P2 被确认为两个**不同的故障**。

例：MAX1 程序

```
int MAX1(int x, int y) {
    int mx = x;
    if (x > y) { mx = y; }   // 输入(3,5)输出3，失效
    return mx;
}
```

- 程序员 A：修第 3 行 `x>y` → `x<y`（MAX2），测试通过，断定第 3 行是 Fault；
- 程序员 B：修第 2、4 行赋值（MAX3），测试也通过，断定第 2/4 行是 Fault；
- MAX2 与 MAX3 **语法不同但语义等价**！

核心结论 ⭐

- 任意程序可构造**无穷多个**与其语义相等但语法不同的程序 → 可定义无穷多个 Bug → 程序修复收敛具有很大不确定性。
- 降低不确定性的工程化方法：**极小化修复**（尽可能少修改代码）。
- 任何需求规格理论上存在无穷正确实现 → 给测试、定位 Bug、调试修复带来极大挑战。

3.4 Bug 的非单调性

基本概念 ⭐

- 单调性：函数值随自变量同向变化；非单调性：变化方向不一致。
- 程序差异比较：若 P\P1 ⊂ P\P2，称 P1 修改小于 P2（P1 <_P P2）。
- **正向测试非单调性**：P(t) 通过，P1 <_P P2，若 P1(t) 失效但 P2(t) 通过 → 测试 t 非单调；
- **反向测试非单调性**：P(t) 失效，P1 <_P P2，若 P1(t) 通过但 P2(t) 失效 → 测试 t 非单调。

例：Mean 系列程序

- Mean（正确，i=0）：t1=[3,4,5] 通过，t2=[4,3,5] 通过；
- Mean1（i=1）：t1 失效，t2 失效；
- Mean2（i=1 且 L−1）：t1 失效，t2 通过 → **修复偏离正确方向更远（越修越坏）**。
- 结论：**单调性和非单调性依赖测试集的构建**。

软件复杂性带来的非单调性

- 代码层面：一个 Fault 可能涉及多行不连续代码；
- 外部引用：修改外部引用引发连锁反应；
- 非单调性现象：“越修越坏”，失效测试数量增加。

3.5 Bug 间的干涉性

干涉定义 ⭐

对于含两个故障 f1、f2 的程序 P，若存在测试 t，使得 t 在 P、仅含 f1 的 P1、仅含 f2 的 P2 上表现各不相同，则称 f1 和 f2 关于测试 t 相互干涉。

干涉类型 ⭐

类型	定义	例子
相长干涉	f1、f2 同时存在导致 t 失效，单独存在时 t 通过	数据读错 Bug + 数据写错 Bug 同时存在导致崩溃

类型	定义	例子
相消干涉	含 f1 的程序 t 失效，加入 f2 后 t 不再失效	一个 Bug 导致运行慢 + 一个 Bug 导致提前退出，同时存在反而正常运行
混合干涉	同一对故障在不同测试上分别呈现相长/相消	测试 t1 相长、测试 t2 相消

> 例：原始程序 P，测试 t1(a=0,b=0) 预期 −1：f1 单独注入输出 −1（通过），f2 单独注入输出 −1（通过），f1+f2 同时注入输出 6（失效）→ 相长干涉。 测试 t1(a=5) 预期 2：f1 输出 3（失效）、f2 输出 1（失效）、同时注入输出 2（通过）→ 相消干涉。

- 💡 练习：针对 MAX、Mean 程序构造相长/相消/混合干涉（含测试集和故障）。

第 4 部分：随机测试方法（3.1 章）📌

> 复习文件（模糊测试设计）的核心方法论基础：随机抽样、自适应筛选、引导反馈、变异生成。

4.1 简单随机测试

概述

- **随机测试**：最简单的多样性测试方法，测试数据通过随机抽样生成，通常基于概率分布，可模仿用户使用场景。
- 优点：低成本生成大量测试、适用性广、易用性高；
- 缺点：缺陷检测效率瓶颈严重，后期能力提升困难（需结合程序分析等方法改进）。

定义 ⭐（复习关联：随机生成）

针对输入空间 Ω，随机测试以**概率分布 F** 进行随机抽样获得测试集 T：

- 默认 F 是均匀分布；
- 实际应用中通常采用**有放回抽样**以简化实现。

应用示例

- **Triangle**：随机生成三个 1~100 的整数 a,b,c；
- **登录系统**：随机生成邮箱（正则/faker）+ 随机密码（大小写字母数字，≥8 位）模拟登录；
- **人脸识别系统**：随机组合人脸质量/分辨率/光照/角度；

- **高级策略**：非均匀分布（概率分布函数生成），如男女面孔概率 0.4/0.6、年轻老年 0.3/0.7、浅色深色皮肤 0.2/0.8。程序的输入/执行/输出空间往往**非均匀**。

随机测试注意点（课后思考）

- Triangle：输入看似均匀，但执行空间和输出空间非均匀；
- NextDay：侧重边界数据（月份、日期、年份越界）；
- MeanVar：可用正态、指数、二项、泊松分布生成数据；
- 超市系统：数据输入 + 打乱操作流程；办公软件：多输入类型 + 多用户协作冲突；鸿蒙小艺：语音/文字指令 + 多用户 + 地理位置。

> 核心观念：**随机不随意**——多样化输入数据、多样性操作交互、多用户并发模拟、智能场景拓展。

4.2 自适应随机测试 ART

动机

随机测试效率较低，命中故障模式的概率取决于软件故障率。**软件失效的故障模式**是指程序故障导致输入域的**连续区域**发生故障——可利用该信息指导随机测试。

三种故障模式（失效区域形态）⭐

1. **点模式**：故障以不连续方式传播；
2. **带模式**：故障沿一个或多个维度拉长成带状；
3. **块模式**：故障形成输入域局部紧凑连续区域。

ART 设计直觉 ⭐

- 连续故障区域的存在 ⇒ 非故障区域也连续；
- 测试应更均匀地分布在输入域；
- **远离已执行测试的新测试更可能失效。**

> 例：输入域 $0\le x,y\le 10$，故障条件 $x+y>3$，正确条件 $x+y>4$（故障区域 $3<x+y\le 4$）。随机测试可能取 (2.2,2.2) 和 (2.1,2.1) 过于接近 → 应避免新测试与已执行测试过于接近。 ⚠️ **距离的定义是决定 ART 选择策略的关键！**

ART 主要步骤 ⭐（复习关联：种子选择）

1. **初始化**：执行初始测试，收集程序行为信息（输出、执行时间、覆盖率）；
2. **执行测试**：将输入空间分成若干子区域，每个子区域随机选一个初始测试；

3. **反馈控制**：根据执行结果更新输入空间，相似测试归为同一子区域；
4. **选择测试**：用自适应策略选择**距已知故障（已执行测试）最远**的测试；
5. **循环重复**：直到满足测试终止条件。

关键概念

- **执行集**：已执行但未显示失效的测试集合（初始为空）；
- **候选集**：无放回随机选择的测试集合，用于更新执行集；
- **选择策略**：从候选集中选离所有已执行测试最远的元素作为下一个测试。

FSCS-ART 算法 ⭐

固定大小候选集 ART（Fixed-Size Candidate Set）：对每个候选测试计算与已有测试的极小距离，选择**最大极小距离**者执行，其他候选丢弃，重复至终止。例：k=3 候选，选 max(min-dist) 的 c2。

距离度量 ⭐ **（复习关联：跨硬件一致性评估用到编辑距离）**

- **欧几里得距离**：空间距离/相似性；
- **曼哈顿距离**：城市行车距离；
- **余弦相似度**：文本相似度；
- **汉明距离**：等长字符串对应位置不同字符个数（编码/检错）；
- **编辑距离（Levenshtein）**：字符串转换的最小编辑操作数（插入、删除、替换）。

ART 应用

- 登录系统 ART：随机生成 1 个执行 + 随机生成 k 个候选（k 一般 10，必须有限）→ 计算候选与已执行测试极小距离 → 选最大极小距离者 → 循环。
- 人脸识别 ART、Triangle/NextDay/MeanVar 的 ART 策略设计。

4.3 引导性随机测试（符号执行）

定义与目标 ⭐

- **定义**：随机抽样结合**符号执行反馈信息**筛选获得测试集 T；
- **目标**：利用符号执行的路径探索能力优化随机测试效率，提高覆盖率、发现深层次缺陷。

符号执行（Symbolic Execution） ⭐ **（复习关联：测试生成）**

定义：一种静态测试方法，通过构造符号化输入代替具体输入，探索程序路径并生成路径约束条件，最后求解约束得到测试输入范围。

流程：

1. 符号化变量：用符号值代替数值；
2. 模拟程序执行：基于中间语言模拟；
3. 记录路径条件：记录语句序列的符号约束；
4. 求解约束条件：用约束求解器求满足路径条件的测试输入；
5. 生成测试用例。

示例：程序 `x=x+2; y=y+x; if (x>5) {...} if (y>0) {...}`

- PC1: x+2>5 ⇒ x>3（如 x=7）；PC2: x+2>5 ∧ y+x+2>0 ⇒ x>3 ∧ y+x>−2（如 x=4,y=0）。

符号执行四大挑战 ⭐

1. **路径爆炸问题**：n 个独立 if-else 分支有 2^n 条路径，指数级增长。
   - 对策：路径剪枝（静态分析排除不可行路径，如 A0∧B0∧C1 中 x≤5∧y≤10 时 x+y≤20 不可满足）、启发式搜索、抽象解释。
2. **内部结构问题**：指针/引用层级、数组（多维）符号索引、链表/树/哈希等动态结构。
3. **外部函数问题**：操作系统/数据库/库函数（rand() 黑盒）难以建模。
   - 对策：函数摘要、模拟执行、动态信息结合。
4. **约束求解问题**：非线性算术、量词约束超求解器能力。
   - 对策：简化约束、近似求解、多求解器结合。

静态符号执行 vs 动态符号执行 ⭐

内容	静态符号执行	动态符号执行（Concolic）
运行原理	为每条路径生成符号约束，不依赖实际执行	先动态执行记录实际路径和取值，再引导符号执行
路径探索	穷举所有路径，指数增长	借助动态信息减少探索，降低搜索空间
约束求解	每条路径都求解，次数多	用动态取值简化求解
效率	开销大但发现更多潜在故障	效率高，但可能遗漏动态执行未出现的路径

动态符号执行示例（如何求解测试输入）

```
int f(int x) { return 2 * x; }
int h(int x, int y) {
    if (x != y) {
        if (f(x) == x + 10) { abort(); /* error */ }
    }
}
```

- 路径 1：x==y → 不需测试；
- 路径 2a：f(x)==x+10 ⇒ 2x=x+10 ⇒ x=10 → 测试输入 x=10, y=6 可触发 abort（发现错误）；
- 路径 2b：f(x)≠x+10 → x≠10。

> 引导性随机测试总结：动静态结合是测试优化的基本思路；执行实时反馈更有效引导符号执行；随机是固定策略无效时的有益补充。

第 5 部分：等价类测试方法（3.2 章）📌

> 复习文件中的"种子语料覆盖典型域""槽位集合"即等价类划分思想的应用。

5.1 软件等价类划分

基本思想 ⭐

- 输入数据通常是无穷的，全部测试不现实也没必要；
- 将输入空间划分为若干**等价类**，从每个等价类中选取代表性数据作为测试用例；
- 等价类：输入域的一个子集，其中各输入对揭露缺陷所起的作用是**等价**的（某输入能发现缺陷 ⇒ 同类的其他输入也有相同可能）。

等价关系三性质 ⭐

- **自反性**：每个元素与自身等价；
- **对称性**：A 与 B 等价则 B 与 A 等价；
- **传递性**：A~B 且 B~C 则 A~C。

等价类测试基本步骤

1. **确定划分域**：确定程序输入或输出的范围；
2. **划分等价类**：将输入域/输出域划分为等价类；
3. **选择代表性测试**：从每个等价类中选择代表性测试。

划分规则：

- 输入条件规定取值范围/个数 → 1 个有效等价类 + 2 个无效等价类；
- 输入条件规定值的集合或必须满足的条件 → 1 个有效 + 1 个无效；
- 布尔变量 → 1 个有效 + 1 个无效。

示例

- **Triangle**：无效等价类（任一边 ≤0 或 >100）、有效等价类（各边 >0 且 ≤100）；输出域：等边/等腰/普通/无效三角形。
- **NextDay**：year（闰年 Y1/平年 Y2）、month（30 天 M1/31 天 M2/2 月 M3/12 月 M4）、day（D1: 1~28 / D2: 29 / D3: 30 / D4: 31）。
- **成绩计算程序**：输入（考试 0~75、作业 0~25）与输出（A≥70、50≤B<70、30≤C<50、D<30）的等价类 + 无效等价类（分数 >75、负数、实数、字母、总分 >100 等）+ 未指定的输出等价类（输出 E、A+、空）。

等价类测试四种类型 ⭐

类型	覆盖要求
弱一般等价类测试	覆盖每个变量的一个有效等价类（每变量 1 个测试）
弱健壮等价类测试	弱一般 + 无效等价类
强一般等价类测试	覆盖每个变量的每个有效等价类组合（笛卡尔积）
强健壮等价类测试	强一般 + 无效等价类组合

> 例：X1 有 3 个有效+2 个无效等价类，X2 有 2 个有效+2 个无效： 弱一般 3 个测试、弱健壮 7 个、强一般 6 个（3×2）、强健壮 10 个（5×2）。

5.2 划分随机测试方法

动机

- 简单随机测试：全域随机抽样，简单易用但针对性差；
- 等价类划分：基于领域知识和历史经验，**完美等价类划分假设往往不成立**；
- **划分随机测试**：实践中不完美的等价类划分 + 随机抽样。

定义与符号 ⭐

- 划分随机测试：对 K 个子域 D1…Dk，按概率 P1…Pk 分配每个子域的强度，为构建大小为 N 的测试集，随机为子域 Di 生成 Ni = pi·N 个测试。
- 符号：d 输入域大小；m 失效输入数；θ=m/d 总体缺陷率；n 测试总数；σ=n/d 总体抽样率；di/mi/ni/θi 对应子域。

三种度量 ⭐（复习关联：评估准则）

度量	含义	公式
P-度量	测试集至少包含一个失效测试的概率	简单随机 Pr=1−(1−θ)ⁿ；划分 Pp=1−∏(1−θi)^ni
E-度量	失效次数的数学期望	Er=nθ；Ep=Σ ni·θi
F-度量	检测到第一个缺陷所需的测试数量（几何分布）	P(F=i)=(1−θ)^(i−1)·θ

结论要点：

- 当各子域缺陷率与全域相同时，Pp=Pr；
- 最佳情况：一个子域内输入全部失效 → Pp=1；
- 测试可能"浪费"在仅含正确输入的子域（如 d1=1, d2=99，失效全在 D2，Pp=0.08 < Pr=0.15）；
- **P 与 E 关系**：若 Ep ≥ Er 则 Pp ≥ Pr；E-度量比 P-度量更能反映测试方法有效性；
- **F-度量**常用于评估软件可靠性：平均无故障时间 **MTTF**（若 MTTF=1000 次运行，N=3000 次测试，置信度 1−e≈0.9503）。

第 6 部分：组合测试方法（3.3 章）

组合测试简介 ⭐

- 适用：具有多个参数的系统，全组合测试数目指数级增长；
- 目标：**用最少用例覆盖最多参数交互缺陷**；方法简单、对测试人员要求低。

基本概念

- **参数（因素）**：影响软件行为的输入变量或配置选项（浏览器、操作系统、分辨率、网络…）；
- **取值**：每个参数的可能取值（Chrome/Firefox/360…）；
- **组合强度（k-way）**：n=1 单因素（一般不计入）；**n=2 双因素 Pairwise（最常用）**；n=3 三因素；k 越大测试数量指数增长。

组合测试示例

- 3 参数（3×3×2）：双因素组合示例 9 条测试；
- NASA 数据库应用数据：**67% 故障由单个因素值触发，93% 由两个因素组合触发，98% 由 3 个因素组合触发** → 大多数故障由 ≤4 个因素交互触发。

非经典组合测试 ⭐

1. **带约束的组合测试**：参数间存在约束（如"800 不能打国际电话"）。不能简单删除测试（会遗漏 800-Loop、Loop-忙音、800-忙音 等组合），应修改该测试为 本地-800-Loop-忙音 或 长途-800-Loop-忙音；也可显式标注无效组合（如不允许"国际-800"，* 通配符）。
2. **种子组合测试**：要求最终测试集包含给定种子测试（如 本地-800-Loop-成功），合并策略往往不是最优。
3. **可变强度组合测试**：不同因素间覆盖强度不同（如呼叫类型-计费方式-连接类型要求 3-way，其余 2-way），保证覆盖率同时降低成本。

💡 练习：办公软件字体配置（中文字体、西文字体、字形、字号、颜色、下划线等）的双因素组合；页面配置带约束双因素组合。

第 7 部分：边界故障假设（4.1 章）

7.1 黑盒边界故障假设

基本思想 ⭐

- 软件系统在处理**边界值**时更容易出错（边界是状态/条件分界线，逻辑变化关键点）；
- 开发人员更关注常规输入输出，边界处理易疏忽（数组越界崩溃）；
- 边界故障假设测试主动探索边界情况，验证边界值及其附近表现。

边界定义 ⭐

- 输入域 D 划分为子域 D1…Dn，Di 有最小值 mini 和最大值 maxi，+ 表示略大于、− 表示略小于；
- **正向边界故障假设**：∪ {$mini^+$, $maxi^-$}（边界内侧）；
- **负向边界故障假设**：∪ {$mini^-$, $maxi^+$}（边界外侧）；
- **边界故障假设**：∪ {$mini^+$, $maxi^-$, $mini^-$, $maxi^+$}。

边界的类型

- **数值范围边界**：最小、最大、略小、略大（1~100 → 1、100、0、101）；
- **字符长度边界**：最小长度、最大长度（3~20 → 3、20、2、21）；
- **业务领域边界**（办公软件边距/纸张大小、人脸识别光照/角度/分辨率/年龄、鸿蒙小艺）。

示例

- **Triangle**：min^-=0 →“无效输入值”；max^+=101 →“无效输入值”；min=1、min^+=2、max^-=99、max=100 结果不能确定（取决于其他两边）。

- **NextDay**：显性边界 year 1900/2050、month 1/12、day 1/28/29/30/31；**隐性边界**：不同月份日数区别，尤其 2 月与闰年平年。
- **MeanVar**：单值输入、两相同值（最小非零方差）、两不同值、最大允许值、超范围值。

基本步骤

功能理解 → 变量识别 → 明确取值范围 → 找出边界点 → 考虑特殊情况；单变量测试用例 + 多变量边界值组合。

偶然正确（Coincidental Correctness）⭐

边界位置 ①：x^+ 观察不到失效但 x^- 能；②：x^+ 和 x^- 都观察不到失效（S 与 P 分界线重叠/偏差方向未知）；③：x^+ 能观察失效但 x^- 不能。

- 不知道偏差方向 → **边界值成对出现**（x^+ 与 x^- 都要测）。

7.2 白盒边界故障假设

概述

- 黑盒以**需求规格 S** 分界线为基线；白盒以**程序实现 P** 分界线为基线（分界点选择问题）；
- 白盒边界故障假设主要关注**代码中的判定决策点**（条件语句、循环语句、逻辑操作符），值往往是逻辑真假值；
- 人类视角：代码；机器视角：计算。

示例

- **Triangle**：第 2 行 `(1<=a<=100 and ...)` 真假边界；第 4 行 `(a+b>c and ...)` 真假边界；
- **MeanVar**：循环边界（数组长度 0/1/≥2）、数组下界 0 起始不越界、两个循环次数一致、X[i] 与 mean 数值大且接近时的浮点稳定性。

计算稳定性 ⭐

- 线性代数不稳定性（接近奇点、小特征值）；微分方程数值算法（舍入误差增长、蝴蝶效应）；算法对波动的处理（抑制或放大）。
- **浮点计算 ≠ 实数运算**：a+(b+c)=(a+b)+c 在浮点下不一定成立（不同舍入模式）。
- 前向误差：结果与解的差值 $\Delta y=y'-y$；后向误差：最小 Δx 使 $f(x+\Delta x)=y'$；混合稳定性。
- **方差计算稳定性**：利用平移不变性 $Var(X-K)=Var(X)$，K 越接近平均值结果越准确。
- 工程任务：选择稳健的数值算法；数值不稳定性分类：数学问题引起（重新定义需求）/ 工程实践引起（改进程序设计）。
- 应对策略：**检测 → 分类诊断 → 改进**。

第 8 部分：变异缺陷假设（4.2 章）📌

> 复习文件中的"种子变异策略"直接采用变异思想（对种子音频做变异生成非预期输入）。

8.1 变异分析基本概念

概述 ⭐

- **变异故障假设测试（变异测试/变异分析）**：对测试集有效性和充分性进行评估的技术；
- 评价测试集/测试方法的缺陷检测能力，辅助构建更强测试集；
- 通过**微小代码语法修改**（注入变异体）检查测试集能否检测代码缺陷；
- 故障假设策略谱系：领域故障/通用故障（人工构造 vs 自动构造）、模糊测试（随机策略）/变异测试（系统策略）。

两大基本假设（DeMillo 1978）⭐

1. **熟练程序员假设**：熟练程序员写的有缺陷代码与正确代码非常接近 → 为简单代码修改注入策略提供基础；
2. **耦合效应假设**：如果测试能检测单一缺陷，则也能检测耦合的复杂缺陷 → 为优先考虑单一变异提供理论基础。

变异体 ⭐

- 变异体：对原程序做微小语法修改得到的程序（符合语法规则、可编译运行）；
- 例：`for(i=0; i<10; i++)` 变异为 `i<=10`、`i--`、`i=1` 等；
- 期待测试 t 对变异体 ρ' 与原程序 ρ 行为不一致。

变异体检测 ⭐

- **杀死（Kill）**：存在 t∈T 使 ρ' 与 ρ 行为不一致，称 T 杀死 ρ'；
- **存活变异体**：t 通过（行为一致），变异体对 t 存活；
- **变异分数（Mutation Score）**：被杀死的变异体比例（或数量），评估测试能力的重要度量；**变异分数依赖变异体，也依赖测试集。**
- **等价变异体**：语法不等价但语义等价（对任意输入行为相同），如 `if(x!=5){if(x>5)}` 与 `if(x!=5){if(x>=5)}`；完全等价变异体难以判定，实践中用足够大的 T 做充分性判定。

变异算子 ⭐（复习关联：变异策略）

- **值变异**：改变参数或值（x=5 → x=10）；
- **语句变异**：增/删/替换语句或改顺序（total=x−y → total=x+y）；
- **决策变异**：变逻辑或算术运算符（if(x>y) → if(x<y)）。

弱杀死 vs 强杀死 ⭐

- **弱杀死**：变异体的**中间状态**与原程序不同（弱变异分析）；
- **强杀死**：变异体的**输出结果**与原程序不同（强变异分析，即传统变异分析）。
- 例：MeanVar `for(i=1;...)`，输入 X=[4,5,3],L=3 分析弱/强杀死。

领域拓展

- 系统级变异：接口变异、功能变异、性能变异；
- 复杂领域：面向对象程序、区块链系统、人工智能系统。

8.2 变异测试优化技术

挑战 ⭐

- 作用：执行变异评估测试集质量；挑战：**大量变异体带来高昂成本**。
- 一般来说：构造变异体成本较小，**执行变异体成本较大**。
- 例：Triangle 第一个分支条件 `not(1<=a<=100 and 1<=b<=100 and 1<=c<=100)` 就能产生大量变异体（改变比较符、and/or 互换、等号替换…）。

优化目标 ⭐

给定初始程序 P、变异体集合 M，优化得到较小的变异体集合 M'，使得对不同测试集 T，T 对 M' 的变异分数近似 T 对 M 的变异分数（或评估能力不比 M 差）。

三种优化方法 ⭐

1. **变异算子选择**：从经验上相似的变异算子中选择**代表性**变异算子（选择强变异算子）；
2. **变异体随机选择**：对相似变异体均匀随机抽样；
3. **变异体聚类抽样**：对变异体聚类分析后随机抽样。

重要概念

- 变异体相似性/强弱：m1 更难被杀死（t 杀死 m2 必杀死 m1，即 m1 被杀死蕴含 m2 被杀死，则 m1 强于 m2）；相似则冗余；
- 代表性变异体、冗余性变异体、真实性变异体；
- J. Offutt 对早期 Fortran 22 个变异算子系统性研究，提出五大类基本变异算子。

第 9 部分：逻辑故障假设（4.3 章）

9.1 逻辑测试基础

逻辑故障定义

- 软件逻辑故障：程序代码中的逻辑错误或不当设计，导致程序无法按预期执行或输出异常行为；
- 特点：通常不导致崩溃，但输出错误结果；
- 成本：n 个变量的逻辑公式需 2^n 个测试枚举，工程上往往不可行；
- **逻辑故障假设是特殊的故障假设，属于变异故障假设的一种。**

逻辑基础

- 谓词逻辑：个体词、谓词、量词、逻辑运算符；布尔逻辑：布尔变量 + 逻辑运算符；
- 运算符：AND（· 可省略）、OR（+）、NOT（上划线/¬）；例：a OR (b AND NOT c) = a+b¬c。

逻辑故障类型（基础）⭐

- **文字非故障**：¬a+b¬c、a+¬b¬c、a+bc（文字被替换/取反/删除）；
- **连接符故障**：ab¬c、a+b+¬c（∧/∨ 互换）；
- **括号符故障**：(a+b)¬c；
- **0 固化故障**：0+b¬c、a+0¬c、a+b·0；
- **1 固化故障**：1+b¬c、a+1¬c、a+b·1。

测试条件与测试点 ⭐

- 给定逻辑表达式 s 及其故障表达式 m，测试对 t、t’ 满足：(1) s(t)=m(t) 且 s(t’)=m(t’)；(2) t 与 t’ 仅相差一个输入值 → 称为**测试条件**，差异点称为**逻辑测试点**。
- 例：原式 a+b¬c 与 0 固化故障 0+b¬c，[0,0,1] 和 [1,0,1] 是测试条件，第一个输入是测试点。
- 同一故障的测试条件/测试点不唯一；不同故障的测试条件/测试点可能相同。
- 测试条件用于差分分析，测试点代表逻辑敏感点。

逻辑测试策略

- 目标：构建**最小测试集**检测尽可能多逻辑故障；
- 思想：采用测试点分析每个变量值单独影响结果；以 ab(cd+e) 为例，单文字测试条件只需 **2n 个测试**；
- 单个测试可能满足多个测试条件（极小化策略：每条件至少一个测试满足，选择满足多个条件的测试减少规模）；
- 挑战：某些测试条件可能有多种满足方式（不确定性），需强制执行某些条件。

逻辑范式 ⭐

- **析取范式（DNF）**：有限个简单合取式的析取（满足一个子句为真即可）；
- **合取范式（CNF）**：有限个简单析取式的合取；
- DNF 易于分析测试条件和测试点（当其他子句为假、只有一子句为真时，该子句每个输入都是潜在测试点），但**转换成本高**。

逻辑测试基本步骤

1. 程序符号执行构建谓词逻辑约束；
2. 谓词逻辑实例化转换为布尔逻辑；
3. 对布尔逻辑进行故障假设分析；
4. 构建测试条件，生成极小化测试集检测逻辑故障；
5. 将测试条件还原为程序输入，实现程序逻辑测试。

挑战

- 工程实际与谓词逻辑的偏差、谓词逻辑与布尔逻辑的偏差（变量间潜在约束：c="高度>5" 与 e="高度<4" 不能同时为 1）；
- 极小化挑战：可能遗漏可行测试、选择不可行测试条件；
- 例：TCAS II 航空防撞系统规范（5~14 个布尔变量，检查变量间依赖）。

9.2 逻辑故障结构

逻辑故障分类（10 种故障类型）⭐

- **操作符故障**：①操作符引用故障（ORF，$\vee \leftrightarrow \wedge$）；②表达式否定故障（ENF）；③变量否定故障（VNF）；④关联转移故障（ASF，省略括号）；
- **操作数故障**：⑤变量丢失故障（MVF）；⑥变量引用故障（VRF）；⑦子句合取故障（CCF，$c \rightarrow c \wedge c'$）；⑧子句析取故障（CDF，$c \rightarrow c \vee c'$）；⑨0 固化故障（SA0）；⑩1 固化故障（SA1）。

检测条件（布尔差分）⭐

- 定义：逻辑 P 的故障 P' 能被检测的条件是测试 t 产生 P 值与 P' 值不同，即 $P(t) \neq P'(t)$，称为测试条件；
- 表达：$P \oplus P'$（$\oplus$ 为异或），记为 d 类型；
- 例：$S=p \wedge \neg q \vee r$，变量 q 的否定故障：$d_SVNF = (p \wedge \neg q \vee r) \oplus (p \wedge q \vee r) = p \wedge \neg r$；
- 布尔差分示例：初始表达式 $A \wedge (B \vee C)$，$d_\bar{A}A = A \wedge (B \vee C) \oplus \neg A \wedge (B \vee C) = B \vee C$ 等。

故障类型蕴含关系（**Kapoor**）⭐

- 检测条件顺序：dVNF → dVRF → dENF；
- 故障关系：VNF ≥f ENF；MVF ≥f VNF；VRF ≥f VNF；CCF ≥f VRF；CDF ≥f VRF；CCF ≥f SA0 ≥f VNF；CDF ≥f SA1 ≥f VNF；
- **Kapoor 结构的局限性**：忽略了布尔规范变异体可能等价，导致若干蕴含关系不成立（VNF≥ENF、MVF≥VNF、SA0≥VNF、SA1≥VNF、CCF≥VRF、CDF≥VRF 均不成立）。

完整逻辑故障联合蕴含结构

- 联合蕴含：如果任何测试检测到对 F 的 δ1 和 δ2 的任何错误实现，能保证检测到 δ3 的任何错误实现，则 (δ1 ∪ δ2) ≥f δ3；
- **核心故障类型**：ASF、ORF、CCF、CDF、ENF；
- **非核心故障类型**：SA0、SA1、VNF、VRF；
- **优先级排序**：CCF 和 CDF 优先于 ASF、ORF 和 ENF。

第 10 部分：Web 环境搭建（实验课，基于 Java）

> 实验课操作手册，用于搭建 Selenium + Java Web 自动化测试环境。按步骤操作即可。

1. 下载配置 JDK 1.8

- 下载安装 JDK 1.8，配置环境变量；用 `java -version`、`java`、`javac` 验证。

2. 下载配置 Chrome 和 ChromeDriver

- 查看 Chrome 浏览器版本 → 下载对应版本驱动（chrome-for-testing 站点）→ 解压保存路径。

3. 下载配置 Maven

- 下载解压（如 E:\Tools\Maven\apache-maven-3.8.1）→ 配置 MAVEN_HOME 环境变量 + Path 添加 `%MAVEN_HOME%\bin` → `mvn -version` 验证；
- 配置本地仓库：settings.xml 的 localRepository 节点；
- 配置镜像：阿里云仓库 mirror（mirrorOf=central）；
- 配置 JDK：profiles 节点激活 jdk-1.8（compiler source/target 1.8）；
- `mvn help:system` 测试配置。

4. IDEA 创建 Maven 项目

- IDEA 全局设置 Maven 主路径、用户设置文件（settings.xml）、本地仓库；
- 配置 pom.xml 依赖：selenium-java、selenium-api（4.0.0）、junit-jupiter（5.0.0/5.0.1）、commons-io（2.11.0）；
- 编写 Selenium 测试代码：ChromeDriver 路径设置、ChromeOptions（–remote-allow-origins=*）、driver.get(url)、隐式等待、截图方法（TakesScreenshot + FileUtils + 时间戳命名）。

第 11 部分：期末复习——模糊测试设计（结合复习文件）📌⭐

> 以下内容根据《软件测试-复习.pdf》整理。复习文件给出了一道完整的**模糊测试设计大题**（车载语音操作系统 VOS 的语音识别模型鲁棒性检测），要求至少包含：**1 种种子选择策略、3 种种子变异策略、2 种测试评估准则**。这是期末考试的典型题型，务必掌握完整答题框架。

11.1 题目背景与要求

背景：VOS（语音操作系统）为车内环境提供语音交互服务（唤醒、语音识别、语义理解），可操作车设车控、地图导航、音乐多媒体、系统设置、空调、天气查询、日程管理、闲聊对话。唤醒词即可使用，一步直达功能，保障行车安全。

任务：设计一种基于模糊测试的方法检测语音识别模型的鲁棒性。网络为**黑盒**（内部结构不可见），可运行于不同硬件环境。

提示：考虑行驶环境的差异和环境的影响。

硬性要求：至少 1 种种子选择策略 + 3 种种子变异策略 + 2 种测试评估准则。

11.2 答题框架（对应复习文件结构）

① 测试目标与假设

- 被测对象（SUT）：车载语音识别模型（黑盒），接口：输入音频 a → 输出转写文本 t（可选：置信度、时间戳、N-best）；
- 部署环境：硬件集合 H={h1…hk}（不同 SoC/CPU/GPU/NPU、不同推理后端/精度）；
- 鲁棒性关注点：
  - 真实行驶环境噪声、混响、回声、通道效应下转写稳定（不产生灾难性错误）；
  - 不同硬件上同一输入应一致或差异可控（避免"同一句话不同硬件识别不同"）。

② 种子语料与标注

- 种子集合 C：录制/合成的车载语音指令音频，覆盖 VOS 典型域：
  - 唤醒词片段+指令（“你好××，打开空调/导航到…/播放音乐…”）；
  - 车控（空调、车窗、座椅、灯光）、导航 POI/地址（数字、地名、多音字）、媒体（歌名、艺人名）、查询（天气、日程）、闲聊（开放域干扰）；
- 基线参考：每条种子音频有参考文本 t*（人工标注或干净录音），便于计算 WER/CER。

③ 模糊测试闭环流程（生成—运行—反馈）⭐

1. 初始化：对每个种子 a∈C，在所有硬件 h∈H 上运行，得到基线转写 t_h(a)；
2. 循环迭代：
   - 按种子选择策略选出一个/一批种子；
   - 用变异策略生成非预期输入 a’（模拟不同车况/噪声/通道）；
   - 在所有硬件上执行：得到 t_h(a’)；
   - 按评估准则计算鲁棒性缺陷分数并反馈：高价值样本入库、触发告警、更新优先级；
3. 输出报告：列出触发样本、变异类型、触发硬件、错误类型（唤醒误触发/漏识别/指令关键词丢失等）。

④ 种子选择策略（至少 1 种）⭐

S1：临界/高风险指令优先（Uncertainty + Safety-Critical Prioritization） 为每条种子计算优先级：

- 指令风险权重：与行车安全/驾驶干扰相关的指令更高（导航、电话、空调除雾等）；
- 语言学脆弱性：含数字、专有名词、地名、多音字、同音词的种子更高；
- 历史不稳定性：多硬件间基线转写已出现差异或少量噪声下易错的种子优先；
- 可形式化为：**Score(seed_a) = α·Risk_a + β·FragileLex_a + γ·D_crossHW_a + δ·ErrHistory_a**
- 解释：ASR 鲁棒性缺陷集中在"关键词/数字/实体"与"安全关键指令"；跨硬件不一致是工程高风险点，应优先深挖。

⑤ 种子变异策略（至少 3 种，给出 4~5 种）⭐📌（对应 4.2 变异思想）

- **M1：行驶噪声叠加（Road/Engine/Wind Noise Mixing）**
  - 将发动机声、胎噪、风噪、雨刷、转向灯、路面颠簸、隧道噪声以不同 SNR 混入语音：**SNR ∈ {20, 10, 5, 0} dB**（轻→重）；
  - 目的：模拟不同车速/路况（城市低速、快速路、高速、隧道）。
- **M2：车内混响与通道效应（RIR/Filter）**

- 施加车厢混响（RIR）或 IIR/FIR 滤波：不同座位位置（驾驶位/后排）、车窗开合、内饰材料差异；
- 目的：模拟车内声学空间变化。

- **M3：回声与多源干扰（Echo + Music/TTS Overlap）**
  - 叠加正在播放的音乐/电台、导航播报 TTS、副驾/后排说话（babble）；控制重叠比例（overlap ratio）与相对响度；
  - 目的：模拟"系统播报时用户插话""车内多人交谈"。
- **M4：语速/音高/口音扰动（Prosody & Accent Perturbation）**
  - 语速 ±10%~20%、音高 ±1~2 半音、发音拉长/吞音（拼接或时域拉伸近似）；
  - 目的：覆盖不同驾驶状态（紧张/急促）、不同说话人差异。
- **M5：片段裁剪与唤醒-指令边界扰动（Truncation/Delay/Jitter）**（补充）

⑥ 测试评估准则（至少 2 种）⭐📌（对应 1.1 测试预言/评估）

- **E1：意图级评估（槽位保持率）**
  - 定义关键词/槽位集合：意图 {打开空调、调高温度、导航到 X、播放 Y…}；槽位 {温度数值、地点实体、歌名/歌手、开/关状态}；
  - 从转写 t_h(a') 抽取槽位（规则/轻量 NLU/正则），计算保持率： **Preserve = 正确保留的关键槽位数 / 应有槽位数**
  - 判定：槽位保持率 < κ（如 0.8）或关键槽位丢失（温度/地点/否定词"不/别"）→ 缺陷候选；
  - 解释：车控场景用户最关心"系统是否执行了正确动作"，关键槽位丢失比一般错字更危险。
- **E2：跨硬件一致性差异（Cross-HW Transcription Divergence）**
  - 对同一变异输入 a'，比较不同硬件输出文本差异：计算任意两硬件转写的**编辑距离**（或归一化距离）；
  - 判定：若 D_crossHW(a') > η（如 0.15~0.25）→ "跨部署不一致风险"；
  - 解释：车载系统常有多硬件版本/多推理后端，跨平台不一致导致同一句话在不同车型行为不同（📌 对应 3.1.2 ART 的距离度量）。

⑦ 反馈与语料池更新（让模糊测试"越测越准"）⭐📌（对应 3.1.3 引导性随机测试）

- 定义"有趣度"：**Interest(a') = λ1·max_h ΔWER_h + λ2·D_crossHW(a') + λ3·(1−Preserve)**
- 满足任一条件即入库：
  - max_h ΔWER_h 创新高；
  - D_crossHW 创新高；
  - 关键槽位丢失；

- 触发异常（空转写、乱码、超时、崩溃、返回空 N-best）。

11.3 复习文件与课程内容的对应关系（答题时灵活引用）

复习文件要素	对应课程内容	章节
模糊测试概念（数据生成、测试运行、结果反馈）	随机测试、模糊测试（Fuzzing）	第 0 部分、4.1
随机生成/抽样种子	简单随机测试（概率分布抽样）	4.1
种子选择策略（优先级/筛选）	等价类划分、ART 反馈控制、测试数据质量	5.1、4.2
种子变异策略（M1~M5）	变异测试/变异算子（值/语句/决策变异）	8.1
评估准则（槽位保持率、编辑距离）	测试预言（o）、测试准则、距离度量（编辑距离/汉明距离）	1.1、1.3、4.2
反馈与语料池更新（有趣度、入库）	引导性随机测试（符号执行反馈）、ART 反馈控制	4.3、4.2
跨硬件一致性	兼容性测试、测试环境 θ 的影响	1.2
黑盒假设	黑盒测试（不考虑内部结构）	1.2

11.4 答题模板（考场速记）

1. **测试目标与假设**：写清 SUT（黑盒语音识别模型）、接口（音频→文本）、硬件集合、鲁棒性关注点；
2. **种子语料**：覆盖唤醒词、车控、导航、媒体、查询、闲聊 + 参考文本标注；
3. **闭环流程**：初始化基线 → 循环（选择→变异→执行→评估→反馈）→ 输出报告；
4. **种子选择策略**：优先级打分公式（风险、脆弱词汇、跨硬件差异、历史错误）；
5. **变异策略 3 种以上**：噪声叠加（SNR）、混响滤波（RIR）、回声/多源干扰、语速音高扰动、裁剪边界扰动；
6. **评估准则 2 种以上**：槽位保持率（意图级）、跨硬件编辑距离（一致性）；可补充 WER/CER；
7. **反馈机制**：有趣度公式 + 入库条件（创新高/关键槽位丢失/异常）。

整理完毕。祝复习顺利，考试加油！